Stock Market Volatility Forecasting using Deep Learning: A Comparative Study of LSTM and GRU Models

Guettara Mohamed Amine – Heddadi Mohamed

Department of computer science, University of Mouloud Mammeri de Tizi-Ouzou, Algeria.

mohamed-amine.guettara@fgei.ummto.dz

haddadimohamed2013@gmail.com

1. Abstract : The main goal of this project is to build a model that predicts the tomorrow's stock market volatility, i used the apple stock market history (history of stock prices) that i got from yahoo finance (i used the yfinance python library to download them[6]) to train the models, i specifically used this stock market because it is famous by its liquidity and it has a massive dataset that is always available.

So, for this project i used to models to compare them, the first one is the LSTM (Long Short Term Memory), and the second one is the GRU (Gated Recurrent Unit), i used these two models because of course they are the best models suited for Time Series Data (this is the kind of data we're dealing with).

The main challenges i faced on this project are the volatility peaks, as the model wasn't doing well with them and giving just small peaks or sometimes ignore it or late reacting to them, and i solved this problem by using the QUANTILE LOSS instead of MSE LOSS, you can find the details on the next chapters of this report.

The two models gave excellent results, as they both did better than the GARCH model that the enterprises still use since 1986[1] with a relative error 9.55% for the LSTM and 9.48% for the GRU (we can see that GRU is the one who did better) and an Improvement of 6 - 10% than GARCH (because GARCH usually give a relative error between 15% and 20% relative error).

And of course i didn't just stop on building the model, as i deployed the model and built an Interactive Graphic Interface that shows the volatility history and the the prediction that represents the tomorrow's volatility.

Keywords : Volatility Forecasting, LSTM, GRU, Deep Learning, Stock Market, Quantile Loss, Optuna, Streamlit.

2. Introduction :

First you think when reading this report is to your self : "why he didn't just tried to predict the prices or maybe the returns of stock, what is even that volatility), i will answer you my

gentleman, first why i didn’t choose the prices, first the prices are not stationary, why it’s not stationary? Because the the std (standard divination) and the mean of the data change with time[5], we know that models like LSTM or GRU can be trained with non stationary data, but it can struggle and do a bad job, because the distribution of the data he sees on the production can change (or sure it will change) from the distribution the LSTM (or the GRU) saw during the training (the LSTM or the GRU can set a borders to the prices and think the prices never go beyond these borders, or maybe draw a false curve out of the window size (since predicting is just draw the curve after the data we gave) that can correspond only to the distribution of the training data and never to a new distribution), and also the prices have no meaning here, as the investors don’t care about the prices but just if the return go up or down (win or lose), so maybe predicting returns is the best choice, now i will tell why i also didn’t choose the returns, we know that the return data or more precise the logarithm of return data is stationary, which is a good sign, but the knockout that let me didn’t choose it is : the return is random, it don’t follow any rule and don’t have a stable pattern, it depends on the human psychology, the news and the events happening and more, for example, we can see how the oil prices go down and up randomly these days because of Trump's declarations about war with Iran. Although i know it’s impossible, but i tried to build a model to predict the return, and the Results were really bad, the model didn’t do well predicting the directions, not just the directions but also the return value, i tried everything (tried many models, changing hyper parameters, adding more features) but still nothing, and that prove that this is an impossible task (i searched on the internet if there is any existing models that work well predicting the return and i found there is no model for now, and that let me know that not my model who did bad work, but the data that don’t have any pattern or rule). So since we can’t predict the return, we can predict how approximately the return volatility gonna be tomorrow, and why the volatility is predicable? isn’t the return random? that’s mean the volatility should be also random? yes the return is random but the volatility has a specific thing that make it a possible to predict, and that think called “the volatility clustering”[5], and it helps predicts the volatility because it indicates that high volatility tends to follow high volatility, and low follows low, creating a "sticky" or persistent pattern. This persistence means current volatility is a strong indicator of future volatility. I decided to predict the volatility of the last 20 days (the tomorrow’s volatility is included), i specifically chose 20 days due its statistical stability, to reduce the noise, to capture the volatility cluster, and also because it’s the balanced choice (not too short and not too long), here’s the volatility equation :

$$vol_t \;=\; std\left(\{r_{t-19}\;,\;\ldots,\;r_t\}\right)\; where\; r_t \;=\; \log\left(\frac{Close_t}{Close_{t-1}}\right)$$

Equation 1 : The volatility Equation

The volatility is very important as it can be used for pricing or risk management (let’s say the tomorrow’s volatility is gonna be very high, that’s mean investors should be cautious, as if they buy stocks today, the prices tomorrow can go extremely high (a big win) or it can go extremely down (a big lose)).

Now to the question, why did i chose a deep learning approach, we know that GARCH exists[1], why do we need to tire ourselves and build a deep learning model for this task, the answer is that GARCH model require linear data, which is not guaranteed on the stock data, this is why

GARCH don’t always give a good results, on the other hand, deep learning models like LSTM or GRU can do an excellent than GARCH on non-linear data, and the relative errors proved we talked about before proved that.

3. Related Work :

GARCH has been the standard for volatility forecasting for almost 40 years[1]. It models variance as a function of past squared returns and past variances. Many extensions exist but the core idea stays the same, Simple, interpretable, and still hard to beat consistently.
LSTM was introduced to fix the vanishing gradient problem in recurrent networks[2]. It uses three gates to control what to keep and what to forget. Many papers tried using LSTM for financial forecasting. Results are mixed. Sometimes it beats GARCH, sometimes not by much.
GRU is a simpler version with two gates[3]. Less parameters, faster training. On short to medium range dependencies it often performs as well as LSTM or better.

What i did differently: i used quantile loss for training and Optuna[4] for hyperparameter search.

4. Data and Features Engineering :

4.1 Data Source :

As i mentioned before, i used the apple stock market data, and chose it due to its liquidity and the availability, and the large size of the dataset that contains the stock market history of more than 20 years, and can be downloaded directly from the yfinance library[6], you need just to give the stock name and the period (easy to use), for this task i picked 16 years of data (January 2010 to March 2026), excluding the days days where the market is close, giving approximately 4000 to 4500 rows of data, the data has initially 5 columns :

- Open: Price at market open.
- High: Highest price during the trading session.
- Low: Lowest price during the trading session.
- Close: Price at market close.
- Volume: Number of shares traded.

Since i mentioned before that the Apple stock market data mostly available, there is low probability to get a NAN value on a row, and if by any chance we got an empty row we can easily do fill forward (fill the messing data with data of the day before).

4.2 Features Engineering :

Of course since we’re dealing with the volatility we need at least calculate the volatility on the dataset, and set it as the target, and to well predict the volatility i calculated many features that

can really help the model learn and well detect the volatility pattern, so from these 5 initial features i got from yfinance, i extracted 17 features grouped in 5 categories like this table shows :

Category	Feature	Definition
Returns and Lags	Return	The profit or loss
	Lag1	Return of the previous day.
	Lag5	Return 5 days Ago
	Lag10	Return 10 days Ago
Price-Based	Ma10	Average price over the past 10 days
	Ma50	Average price over the past 50 days
	Price_vs_ma50	The Price compared to the MA50 : ((Price – MA50) / MA50)
	High_low_range	The Low price compared to the High Price : ((High – Low) / Close_Price)
Volatility	Volatility5	The std of Return over the past 5 days
	Volatility10	The std of Return over the past 10 days
	Volatility20	The std of Return over the past 20 days
	Volatility60	The std of Return over the past 60 days
	Vol_ratio	The ratio of volatility : (Volatility5 / Volatility20)
Technical Indicator	RSI	The Relative Strength Index (The momentum indicator), can be calculated using a python library called “ta”.
	MACD	Measures distance between 12-day and 26-day EMAs to identify trend momentum, and it also can be calculated using “ta”.
Volume	Volume_Change	The Volume change Percentage between a day and a next day.
	Volume_Ratio	The Volume Ratio, represents the Volume divided by the Volume Mean over the past 20 days

Table 1 : The Features after the Features Engineering

And the Target gonna be the Volatility20 in our task, and as i mentioned before, i chose 20 days due its statistical stability, to reduce the noise, to capture the volatility cluster, and also because it’s the balanced choice (not too short and not too long).

4.3 Train and Test Split :

For the splitting, i gave 80% of the data to the train set, and the rest (20%) to the test set, and of course because it's a Time Series Data, i didn't just do a random shuffling because that would break the time order and mixes the future data with the past data, but instead i did just a chronologically split (without any shuffling, just divide the data), and i did the same with the Training set to split it into a Training Set (90%) and a Validation Set (10%).

4.4 Sliding Window :

After splitting the data we need to make the sliding Windows that needed for the LSTM and GRU, and it done setting a window size "W" first, then for each day "t", we take all the previous W days (the day t is included), that series of size W represent the input "X" of each data instance, and set the day "t+1" volatility as the target, which represents the "y" here, at the end we're going to have N – W instances on our Dataset, N represents the number of rows on the dataset after the Features Engineering. The W is treated as a hyper-parameter and was optimized with Optuna, the best values are showed after on the next chapter of this report.

4.5 Feature Scaling :

For the Feature Scaling i was planning to use a Standard Scaler, but because the stock data is full of outliers data, which can make a Standard Scaler Struggle, so the solution is to use a more powerful scaler that can handle outliers better than the Standard Scaler, which the Robust Scaler that use the median value and the IQR values to Normalize the data, the Formula of Robust Scaler is like that :

$$X_{scaled} = \frac{X - \text{median}}{Q3 - Q1}$$

Equation 2 : The Robust Scaler Formula

Where:
Q3 : Represents the Data at position 75% on the Sorted Dataset

Q1 : Represents the Data at position 25% on Sorted Dataset

When i trained the Scaler i used only the training set to train it to prevents the data leakage, or otherwise the scaler can see information about the test set.

5. Model Architectures :

5.1 LSTM (Long Short Term Memory) :

The LSTM is a type of Recurrent Neural Networks (RNN), it has 3 gates : the input, forget, output. These gates control how much the model forget the data and how much it remembers, and this

approach solve the big problem on the RNN which are : the gradient vanishing and the gradient exploding.

On this project, after performing a hyper parameters tuning with Optuna, the final LSTM architecture become like on the next image :

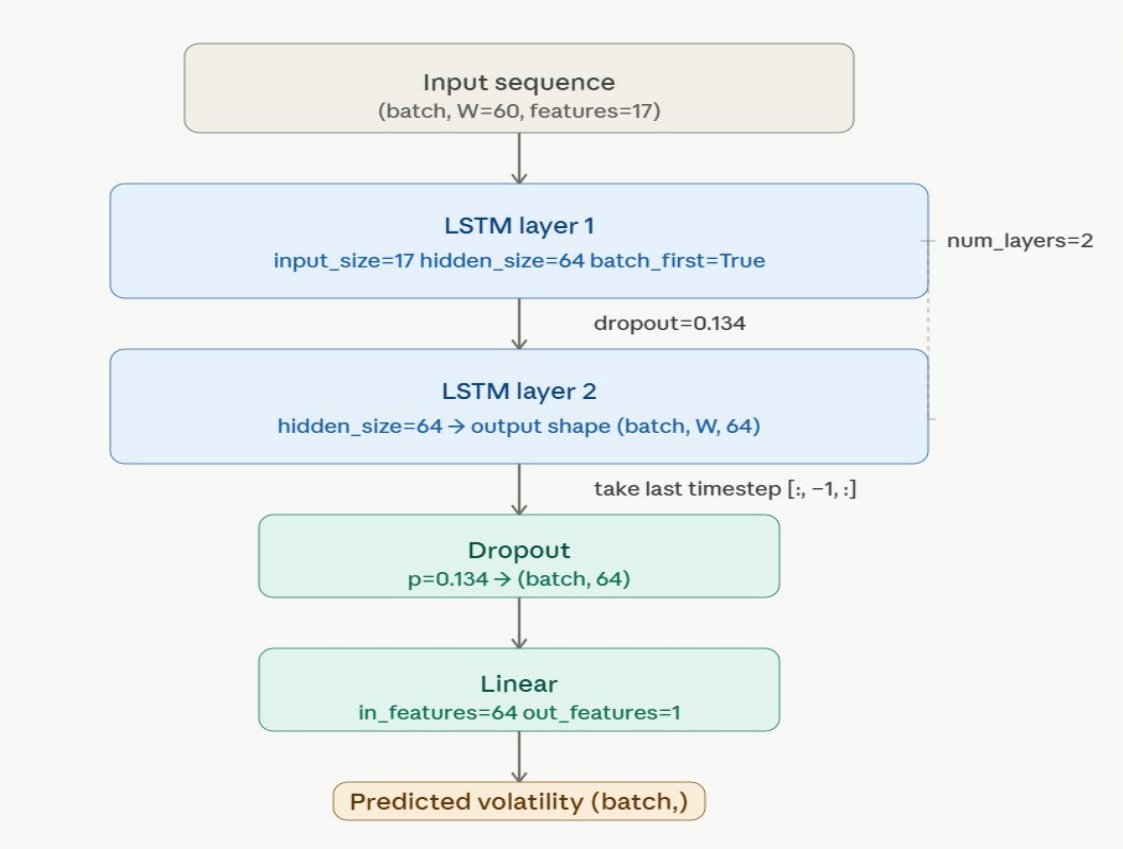


Fig 1 : The LSTM Architecture i used on this project

5.2 GRU (Gated Recurrent Unit) :

The GRU is also another type of Recurrent Neural Networks (RNN), it has just 2 gates, Update and Reset, and because it has fewer gates than LSTM that's mean it's automatically has fewer number of parameters and smaller than the LSTM. And after performing a hyper parameters tuning with Optuna, the final GRU architecture become like on the next image :

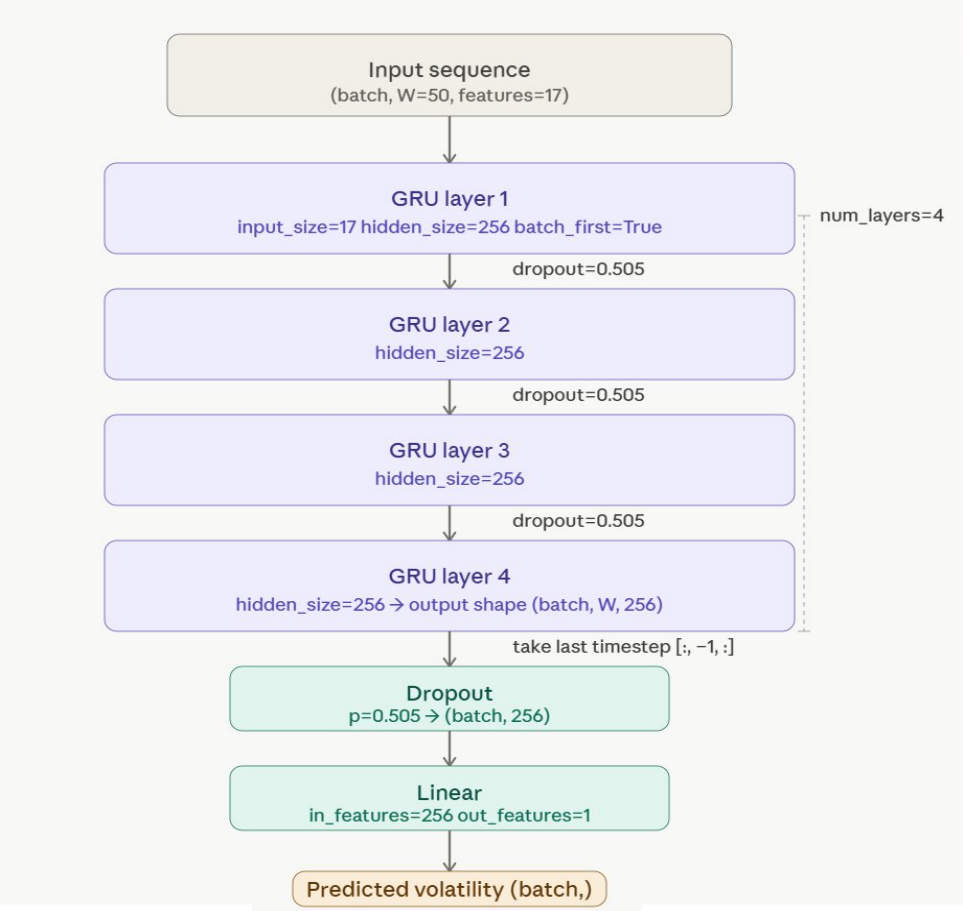


Fig 2 : The LSTM Architecture i used on this project

6. Training Methodology :

6.1 Loss Function :

Since we're dealing with volatility that is a continue real number, that's mean we have a regression task, so the first loss function we think about is the MSE Loss (Mean Squared Error), which is the suited function for many regression tasks. So i tried it but it didn't gave a good results, i'm not saying that MSE didn't go down, but the predictions weren't good, as when i plotted the Actual vs Predicted Curve, i noticed that the model didn't work well on peaks, as the model was giving small peaks, ignore them, or late react to them, i asked myself, why the model is doing that? Why it doesn't do well with the peaks? I thought maybe it's just the hyper-parameters problem, adjusting them can solve the problem? But the model was always struggle with peaks, i said maybe the model isn't giving the peaks any important, maybe if i gave more loss value when the model miss the peak can solve the problem, so i tried the QANTILE Loss, that give more loss if the predicted value is less than the actual value or bigger (depends on the quantile value), since the model is giving small peaks or just ignore them, then we need to give a big loss when the predicted value is less than the actual value, and for that the quantile should be greater than 0.5 (a quantile equal to 0.5 represents the MSE Loss), i initially give 0.65 as quantile, and BAAM, the peak problem

was solved, and the model gave good predictions for the peak. The qunatile loss equation is like on the next image:

$$L_q\left(y,\ \hat{y}\right)\ =\ q\cdot\max\left(y\ -\ \hat{y},\ 0\right)\ +\ (1-q)\cdot\max\left(\hat{y}\ -\ y,\ 0\right)$$

Equation 3 : The Equation of the Quantile Loss

This Graph bellow represents how the quantile value affect the loss, and we should give a quntile greater than 0.5 :

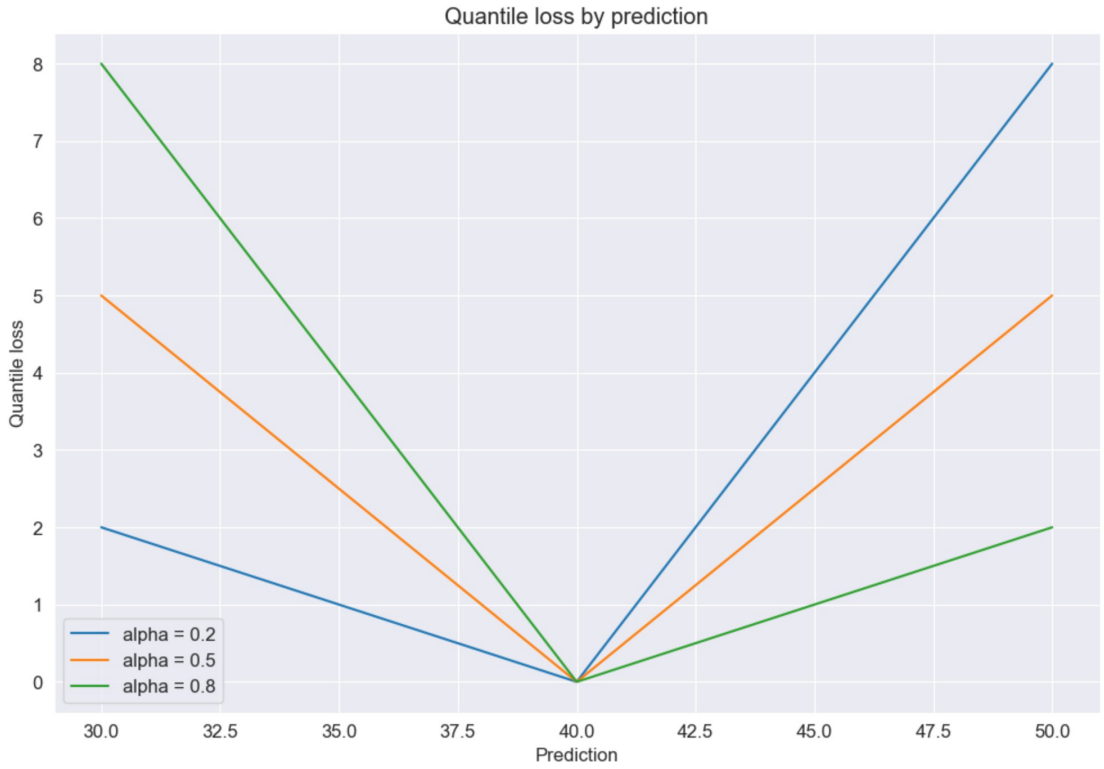


Fig 3 : The Quantile loss value with different quantile values (alpha) where the actual value is 40.0

The quantile is also was treated as a hyper-parameter and was optimized with Optuna, the next Chapter will show the best value for each model.

6.2 Hyper-Parameter Optimization:

So, how did i find the best parameters for these models? i used Optuna[4] to perform a smart hyperparameter search . i didn't choose the traditional Grid Search because it’s just too slow and not efficient, instead i used Optuna because it uses TPE (Tree-structured Parzen Estimator).

What TPE does is it learns from every trial i run and focuses the search on the most promising areas, which is much better than just guessing, i ran exactly 100 trials for each model , and to make things even faster, i used the MedianPruner. This "Pruner" is a life-saver because it kills the bad trials early if they aren’t doing well compared to the others, so i don't waste time or power on them. And to find the best Quantile value when optimizing, i used the quantile loss for

training and MSE Loss for the validation (that's just during the Optuna optimizing, otherwise, on the actual training after finding the best hyper-parameters i used Quantile Loss for both training and validation).

After performing the hyper-Parameters tuning, i got the importance of each hyper-parameter (Optuna automatically give these results), the results are like on the next images :

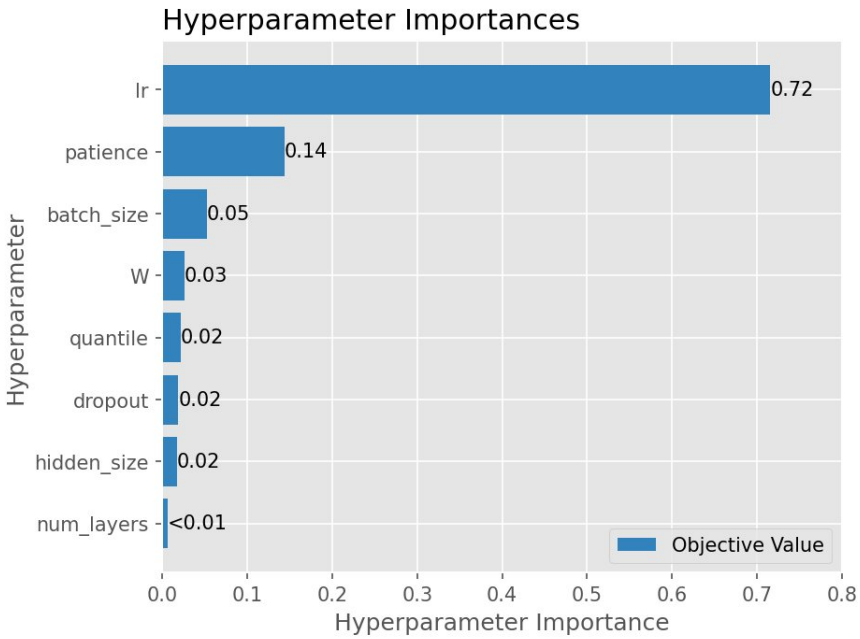


Fig 4 : The hyper-parameters Importance after Performing the Optimization for the LSTM

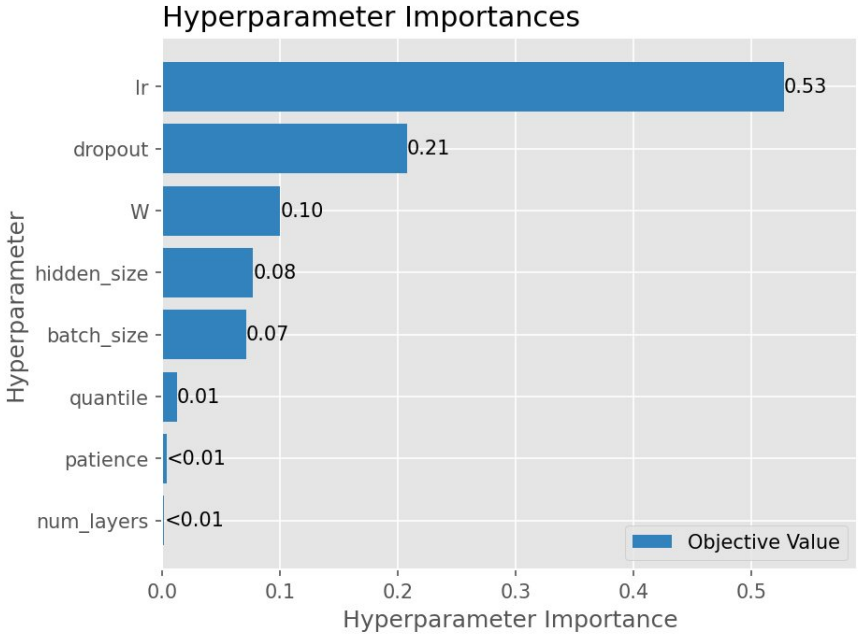


Fig 5 : The hyper-parameters Importance after Performing the Optimization for the LSTM

From the Two images, we can see that the learning is the most important hyper-parameter for both models, and the second more important hyper-parameter is the patience (how much of epochs should pass so the model stop learning if the validation loss never improved) for the LSTM, and the dropout (percentage of neuron to shut down of a hidden layer) for the GRU.

The best Hyper-Parameters for each model is shown on this table :

Hyper-Parameter	LSTM	GRU
Hidden size	64	256
Number of layers	2	4
Dropout	0.13425006919751267	0.5049696839451802
Learning rate	0.0020733744699150854	0.0014233011661517 96
Batch size	48	32
Patience	23	20
W (window size)	60	50
Quantile	0.6176637604495911	0.6413430231856577

Table 2 : The best Hyper-Parameters

6.3 Training Details :

So, for the optimizer, i used Adam, and i also set up a learning rate schedule called ReduceLROnPlateau. This is really smart because it automatically cuts the learning rate by half whenever the validation loss stays the same for 5 epochs. i did this to help the model fine-tune itself when it gets stuck.

One thing that is super important for deep RNNs like the ones i built is Gradient Clipping. i used a max norm of 1.0 because, as i mentioned before, these models can struggle with gradient exploding, and this keeps the training stable. i also used Early Stopping to prevent any overfitting. Since i kept 10% of the training data for validation, i just tell the model to stop training if that validation loss doesn't improve after a few epochs, and i always keep the weights from the best epoch (by take the best parameters as the first parameters, then take the next epoch parameters if the validation loss improved).

The training and validation curves are showed on these images :

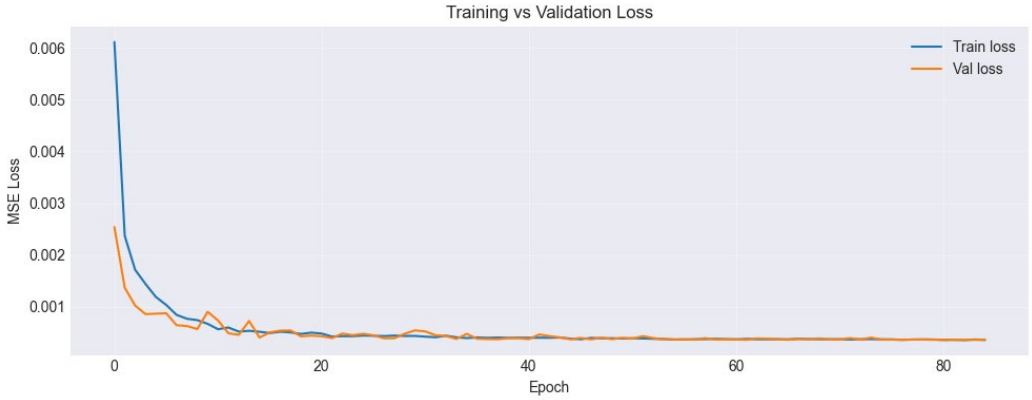


Fig 6 : The training and validation curves of the LSTM

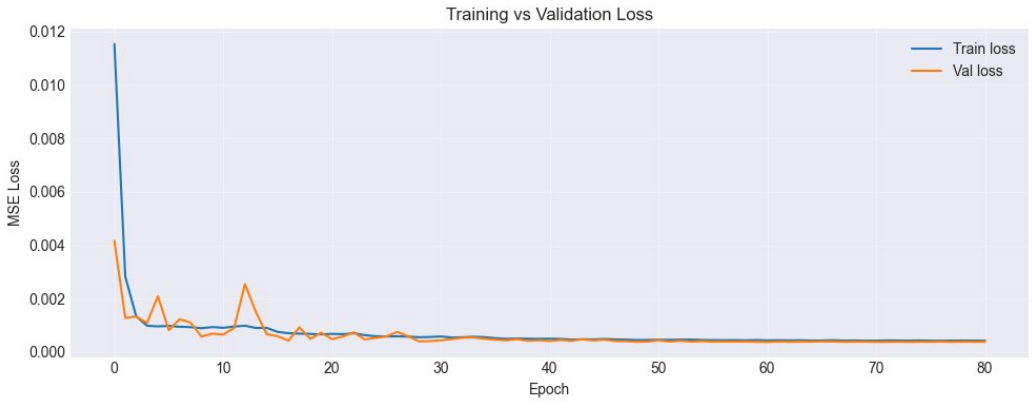


Fig7 : The training and validation curves of the GRU

So from these 2 curves, we can see that they both converge perfectly, The validation loss never starts to diverge from the training loss, which means the model is learning the patterns without any overfitting, we can say that the model generalize well with the unseen data.

7. Results and Discussion :

7.1 Evaluation Metrics :

Now you might ask yourself, how did i even know if my models were doing a good job or just guessing? i will answer you my gentleman : i used two main metrics to find out. The first one is the Relative Error which is the RMSE divided by the mean of the volatility (The True Labels), here's the formula:

$$\mathrm{Re}\,lativeError \;=\; \frac{RMSE}{\mathrm{mean}\,(volatility_{true})}$$

Equation 4 : The Formula of The Relative Error

i used this because it gives a clear percentage of how big the error is compared to the typical volatility values. But the second metric (and this is the real knockout) is the Directional Hit Ratio. This is simply the proportion of days where i correctly predicted if the volatility was going to go up or down. For any investor, this is way more directly useful than just a raw error number.

And of course, i didn't just let my models run in a vacuum. i compared them against two baselines to see if they were actually special. The first is GARCH, which has been the industry standard since 1986. The second is Persistence, where you just assume tomorrow's volatility will be exactly the same as today's. You might think Persistence is a weak baseline, but because of the "volatility clustering" i mentioned earlier (where high volatility tends to follow high volatility) it is actually very hard to beat. Beating it is not trivial at all, but that is the only way to prove a deep learning model is worth the effort and was the right choice.

7.2 Quantitative Results :

And of Course like i said earlier, i need to compare my models with real world models that enterprises use, i putted all the results and differences on this table :

Model	Relative Error	Vs Persistence	Vs GARCH	Directional Hit Ratio
Persistence	9.63%	/	Better	/
GARCH	15 - 25%	Worse	/	/
LSTM	9.55%	+0.08%	Better	56.28%
GRU	9.48%	+0.15%	Better	56.87%

Table 3 : A Table shows the difference between my model and the models used in production

As we can on the table, Both LSTM and GRU beat the Persistence and GARCH, which can confirm that a deep learning model beat the old model GARCH and the final boss (Persistence).

We can also see that the LSTM gave 56.28% on the Directional Hit Ratio, and 56.87% for the GRU, which is a good value since it pass the half, that's mean the model isn't just giving random directions but it thinks. And after performing a search on the internet, i found out that the 56% Directional Hit Ratio is considered very good value in term of volatility prediction and can help on the risk management.

And also from the table we can clearly see that the GRU Model beat the LSTM model, even as the LSTM known by its long memory, but looking at the results, it looks like the volatility clustering doesn't require a long-range but just a short-range which the GRU can handle it very well.

7.3 Predicted vs Actual Volatility :

Now i want to show you the real results of all this work. So for that, i plotted 200 days of my models' predictions against the actual values (Look at Fig 8 and 9). And honestly, The results are beautiful! You can see that the predictions track the actual curve really well. The model is smart enough to catch the calm periods when nothing is happening, but it also stays on top of the turbulent ones when the market goes crazy.

But i have to be honest with you, sometimes there is a little delay of one or two days when a sudden spike hits. You might ask yourself, is this a bug in the code or a problem with the training? i will answer you: absolutely not, This is just the nature of forecasting. You can't really react to a massive event before it actually happens, right? Even the GARCH model has this exact same delay. It's not a mistake, it's just how time series data works in the real world. Despite that tiny lag, the way it follows the trend is just excellent. And Yeah, the models are definitely doing exactly what i wanted them to do!

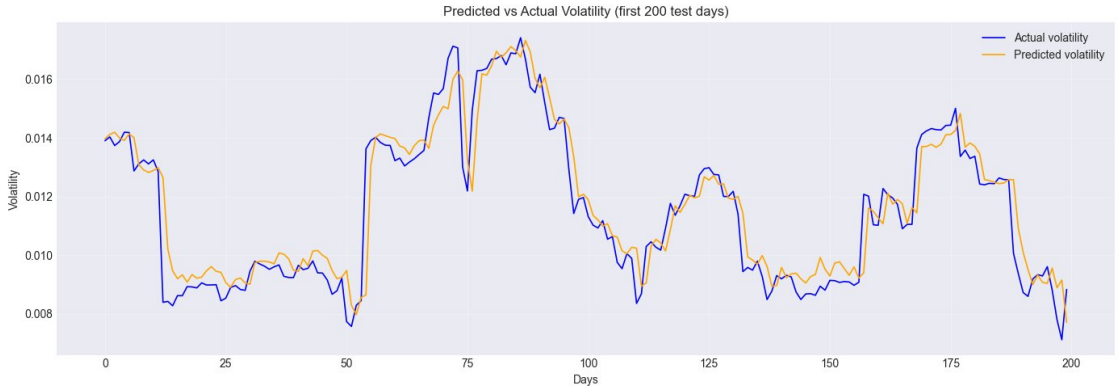


Fig 8 : The predicted vs Actual volatility of the model LSTM

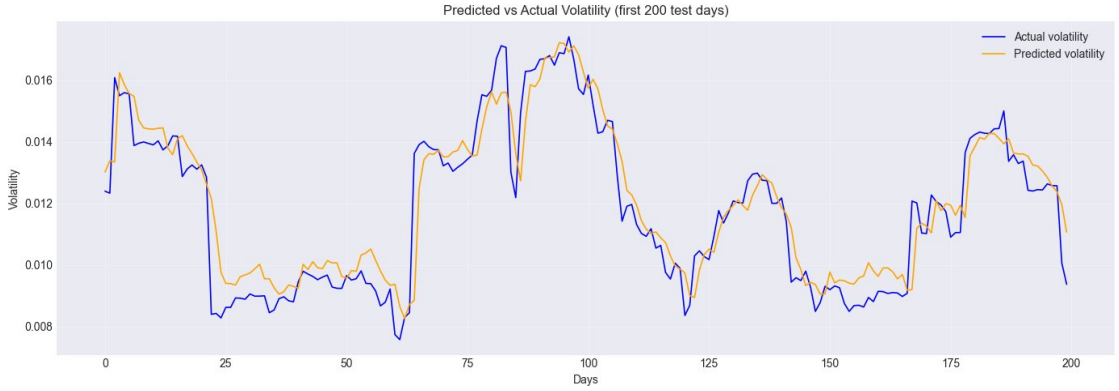


Fig 9 : The predicted vs Actual volatility of the model LSTM

7.4 Deployment and The Web Application :

So, after i finished building and testing these models, i didn't just want them to stay as code on my computer. i wanted to make something practical. So, i built a Streamlit app for live, real-time predictions.

How does it work? i will answer you: the app automatically fetches the latest Apple data directly from Yahoo Finance (using the same library i used for training), computes all the features on the fly, and runs them through the model. i even added a sidebar where you can switch between the LSTM and the GRU models whenever you want to compare them.

The output is very clear, it gives you the predicted volatility, a visual chart, and most importantly, a risk level (Low, Medium, or High). i did this so anyone can understand if the market is safe or too risky at that moment without being a math expert. And just like that, i turned a deep learning project into a live tool that actually works in the real world!

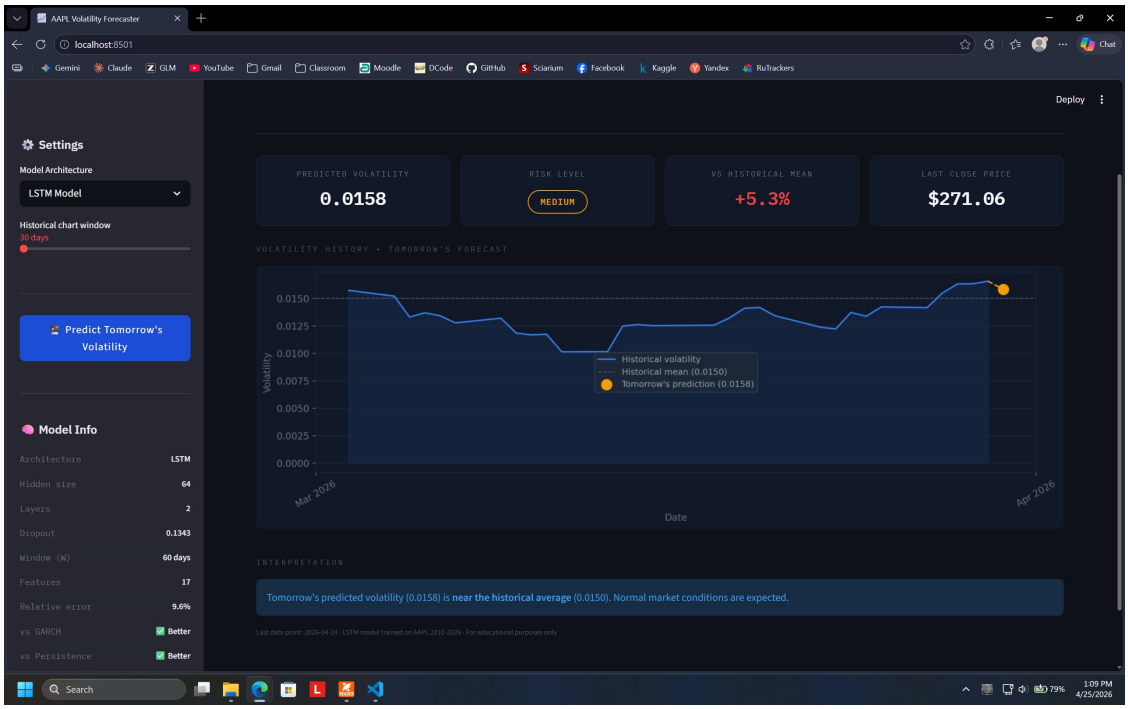


Fig 10 : A screenshot from the web app showing the Prediction of the model LSTM

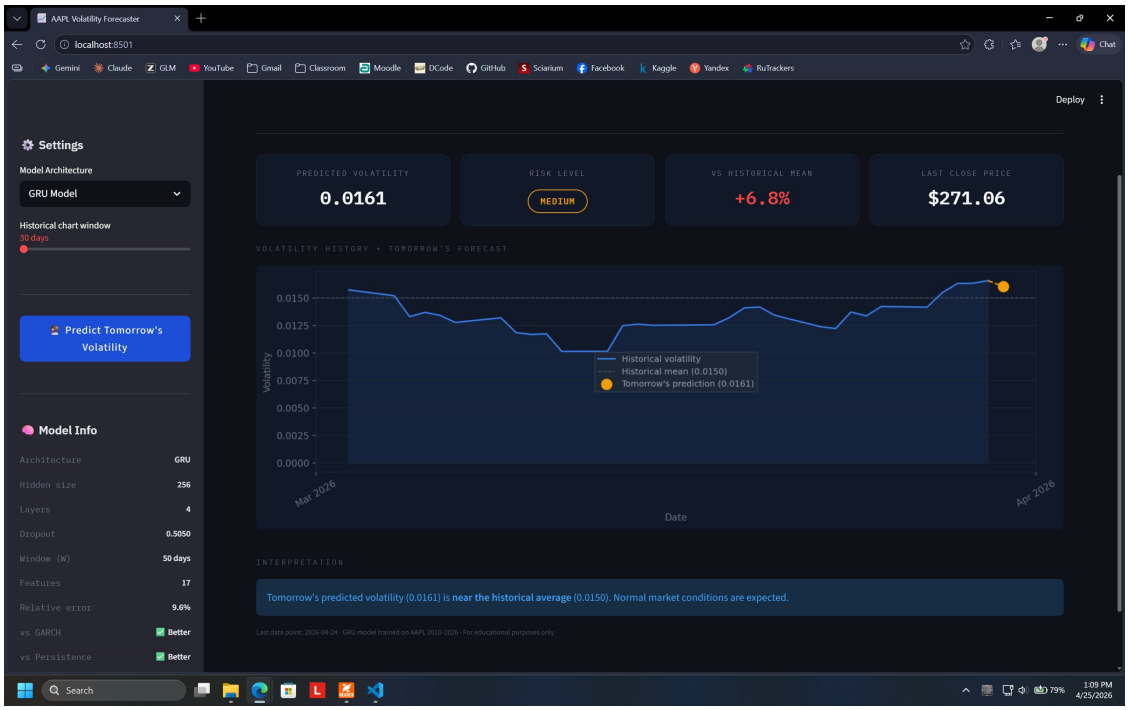


Fig 10 : A screenshot from the web app showing the Prediction of the model GRU

8. Conclusion :

So, The project wasn’t easy, i spent a massive amount of time comparing the LSTM and the GRU to see which one could handle the Apple Stock Market volatility better, and the results are honestly excellent. Both models were strong enough to beat the Persistence baseline and the old GARCH model that everyone still uses. But if i have to pick a winner, the GRU was just a tiny bit better, reaching a 9.48% relative error and a 56.87% directional hit ratio.

But i want to tell you something, the choice between using an LSTM or a GRU wasn't actually the most important part of this project. But it was two critical things, First, when i decided to predict the volatility directly as my target instead of trying to predict the prices, which made the data much cleaner for the model. And second (and this was the real knockout) was using Quantile Loss instead of MSE. That change alone is what solved the peak problem. These two decisions mattered way more than which specific architecture i chose.

So, Do you think i’m done? NO! i’m not done yet! For my future work, i want to try a GARCH-LSTM hybrid where i take the predictions from GARCH and feed them into the LSTM as extra features, kind of like giving the model a "cheat sheet" from the old school methods. i also want to generalize this beyond just Apple Stocks and maybe even add news sentiment using FinBERT to see if the model can react to "breaking news" before the market does. And Yeah, that’s the next level of this project.

9. References :

[1] Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. Journal of Econometrics, 31(3), 307-327.

[2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735 1780.

[3] Cho, K. et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. EMNLP 2014, 1724-1734.

[4] Akiba, T. et al. (2019). Optuna: A next-generation hyperparameter optimization framework. KDD 2019, 2623-2631.

[5] Yahoo Finance. AAPL Historical Data. Apple Inc. (AAPL) Stock Price, News, Quote & History - Yahoo Finance, last accessed 2026.

[6] yfinance. Yahoo Finance market data downloader (Python library). ranaroussi/yfinance: Download market data from Yahoo! Finance's API, accessed April 2026.